PROIECT SCADA

Sistem Automat de Reglare a Iluminării

Bazat pe Senzor de Lumină Ambientală APDS-9930

și Potențiometru Digital X9C103S

Stud. **Andrei BLEJAN**

Platformă de control: Arduino Uno R3

Cuprins

1. Descrierea Proiectului

2. Schema Bloc a Sistemului

3. Componente Hardware

4. Protocolul I2C

5. Arhitectura Algoritmului de Control

6. Arhitectura SCADA

7. Schema Electrică — Tabel de Conexiuni

8. Rezultate și Testare

9. Concluzii

10. Bibliografie

Anexe

1. Descrierea Proiectului

1.1 Obiectiv

Proiectul implementează un sistem de reglare automată a iluminării utilizând un senzor de lumină ambientală, un potențiometru digital și un microcontroller Arduino Uno. Sistemul măsoară nivelul de iluminare ambientală (în lux) și ajustează automat intensitatea unui LED prin intermediul unui potențiometru digital, realizând o buclă de reglare în buclă închisă. Pentru reglarea precisă a procesului neliniar, se vor implementa două metode avansate de control: PID adaptiv cu gain scheduling continuu si un modul de compensare a neliniarităților prin corecție Gamma.

1.2 Funcționalitate

- Achiziție date: Senzorul APDS-9930 măsoară lumina ambientală (lux)
- Procesare: Arduino Uno calculează eroarea și determină comanda pentru reglare
- Acționare: Potențiometrul digital X9C103S ajustează curentul prin LED
- Monitorizare: LCD I2C afișează în timp real valorile măsurate
- Supraveghere SCADA: Interfața Serial permite monitorizare și control de la PC

1.3 Aplicabilitate Practică

- Sisteme de iluminat inteligent (smart lighting)
- Automatizări în clădiri (BMS — Building Management Systems)
- Sisteme de economisire a energiei
- Iluminat adaptiv în medii industriale

2. Schema Bloc a Sistemului

Sistemul este organizat pe trei niveluri funcționale, conform arhitecturii SCADA:

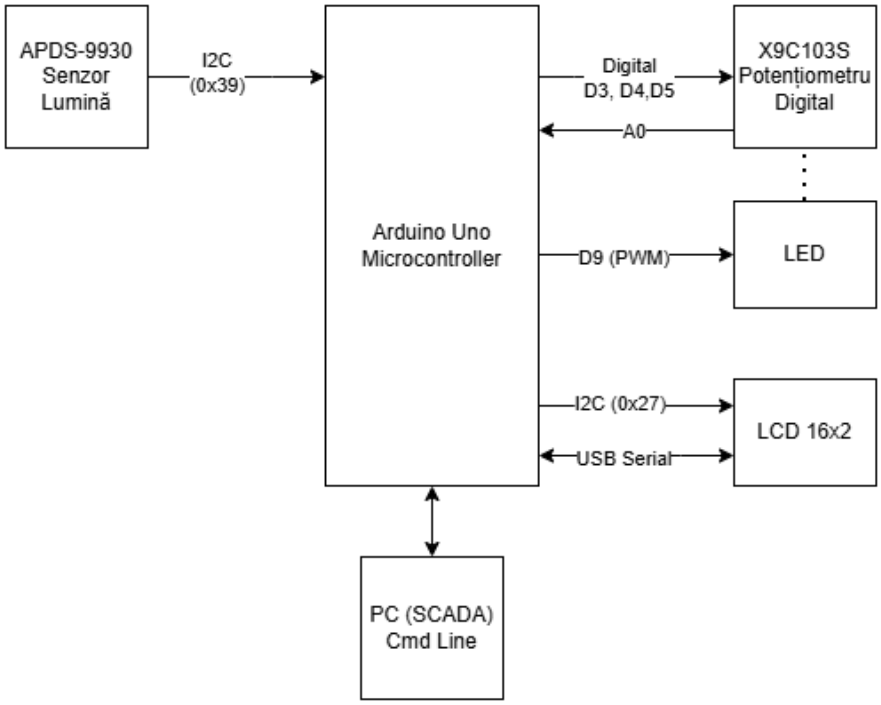


Figura 1: Arhitectura sistemului de control

2.1 Schema Buclei de Reglare

Sistemul implementează o buclă de reglare în buclă închisă (feedback negativ):

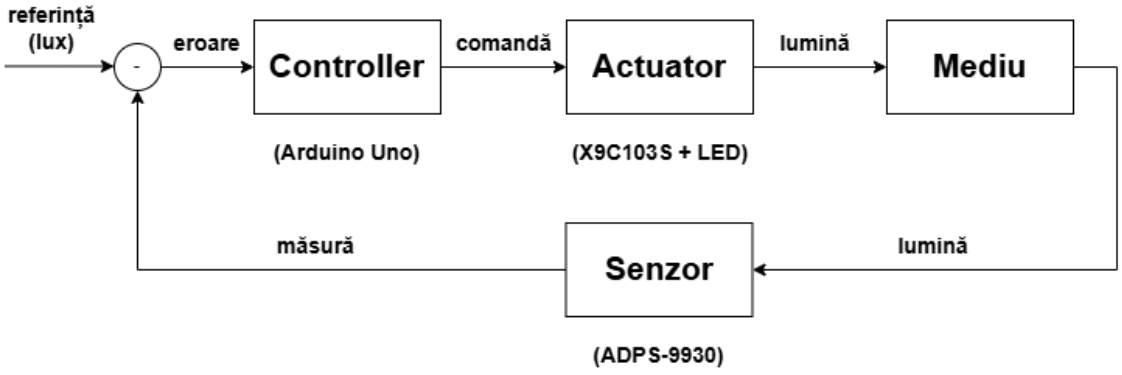


Figura 2: Bucla de reglare a sistemului

3. Componente Hardware

3.1 Arduino Uno R3 — Unitatea de Control

Arduino Uno R3 este placa de dezvoltare bazată pe microcontrollerul ATmega328P, utilizată ca unitate centrală de procesare a sistemului. Implementează algoritmul de reglare, citește senzorul, comandă actuatorul și comunică cu interfața HMI (LCD + Serial).

Parametru	Valoare
Microcontroller	ATmega328P
Frecvență ceas	16 MHz
Tensiune operare	5V
Pini digitali I/O	14 (din care 6 PWM)
Pini analogici	6 (ADC 10-bit, 0–1023)
Memorie Flash	32 KB
SRAM	2 KB
EEPROM	1 KB
Interfețe	UART, SPI, I2C (TWI)
Convertor ADC	10 biți, Vref = 5V, rezoluție = 4.88 mV/pas

3.2 APDS-9930 — Senzor de Lumină Ambientală și Proximitate

APDS-9930 este un senzor digital integrat de proximitate și lumină ambientală produs de Broadcom (Avago). Conține două fotodiode cu filtre spectrale diferite: canalul Ch0 detectează lumină vizibilă + infraroșu, iar canalul Ch1 detectează doar infraroșu. Prin scăderea contribuției Ch1 se obține doar componenta vizibilă, din care se calculează lux.

Parametru	Valoare
Producător	Broadcom (Avago)
Interfață	I2C, adresa 0x39
Tensiune operare	2.4V – 3.6V (3.3V)
Canal Ch0	Fotodiodă vizibil + infraroșu (640 nm)

Canal Ch1	Fotodiodă infraroșu (850 nm)
Domeniu lux	0 – 100.000 lux
Rezoluție ADC	16 biți (0 – 65535)
Timp integrare ALS	2.73 ms – 699 ms (configurabil prin ATIME)
Câștig ALS (AGAIN)	1×, 8×, 16×, 120×
Proximitate	LED IR intern + fotodiodă, domeniu ~100 mm
Consum	110 µA (activ), 2.2 µA (sleep)

Registre principale utilizate:

Registru	**Adresă**	**Funcție**
ENABLE	0x80	Pornire/oprire module (PON, AEN, PEN, WEN)
ATIME	0x81	Timp integrare ALS: t = 2.73 × (256 − ATIME) ms
CONTROL	0x8F	Câștig ALS și proximitate
CH0DATAL/H	0x94-0x95	Date canal 0 (vizibil + IR)
CH1DATAL/H	0x96-0x97	Date canal 1 (doar IR)
PDATAL/H	0x9C-0x9D	Date proximitate

3.3 X9C103S — Potențiometru Digital 10kΩ

X9C103S este un potențiometru digital CMOS cu 100 de poziții (pași) și rezistență totală de 10 kΩ, produs de Renesas (Intersil). Se controlează prin 3 pini digitali (CS, U/D, INC) și are memorie non-volatilă care păstrează ultima poziție la oprirea alimentării.

Parametru	**Valoare**
Producător	Renesas (Intersil)
Rezistență totală	10 kΩ ± 20%
Număr de pași	100 (0 – 99)
Rezoluție per pas	~100 Ω/pas
Tensiune operare	5V
Interfață	3 fire digitale (CS, U/D, INC)
Memorie	Non-volatilă (păstrează poziția la oprire)

Timp setare per pas	~1 µs minim
Consum	0.5 µA (standby), 750 µA (activ)

Pinout:

Pin	Nume	Funcție
1	INC	Increment — front descrescător mișcă cursorul
2	U/D	Direcție: HIGH = spre VH, LOW = spre VL
3	VH	Terminal „High" (conectat la 5V)
4	VSS	Masă (GND)
5	VW	Wiper — ieșirea cursorului
6	VL	Terminal „Low" (conectat la GND)
7	CS	Chip Select — activ pe LOW
8	VCC	Alimentare +5V

Protocol de comandă:

1. CS → LOW (activare chip)
2. U/D → HIGH sau LOW (setare direcție)
3. INC → puls LOW → HIGH (un pas)
4. CS → HIGH (dezactivare + salvare în memorie non-volatilă)

Tensiunea pe wiper:

$$V_WIPER = V_H \times (P / 99)$$

unde P = poziția curentă (0–99), V_H = 5V.

3.4 LED — Actuator de Iluminare

Parametru	Valoare
Tip	LED standard 5mm
Tensiune directă (Vf)	2.0 – 3.2V
Curent nominal (If)	20 mA
Rezistor limitare curent	220 Ω
Conectare	Pin D9 (PWM) prin rezistor

Calculul rezistorului de limitare a curentului:

R = (VCC - Vf) / If = (5V - 2V) / 20mA = 150Ω → se folosește 220Ω (siguranță)

3.5 LCD 16×2 I2C — Interfață HMI Locală

Parametru	Valoare
Tip afișaj	LCD 16 coloane × 2 rânduri
Controller	HD44780 + PCF8574 I2C expander
Interfață	I2C, adresa 0x27 (sau 0x3F)
Tensiune	5V
Backlight	LED, controlabil
Caractere	5×8 pixeli, 8 caractere custom

4. Protocolul I2C (Inter-Integrated Circuit)

4.1 Descriere Generală

I2C (Inter-Integrated Circuit) este un protocol de comunicație serial sincron half-duplex pe 2 fire, dezvoltat de Philips (NXP). Este utilizat pe scară largă pentru comunicația între microcontrollere și periferice la distanțe scurte.

Parametru	Valoare
Linii fizice	SDA (date), SCL (ceas)
Topologie	Bus multi-master, multi-slave
Adresare	7-bit (128 adrese) sau 10-bit
Viteze standard	100 kHz (Standard), 400 kHz (Fast), 3.4 MHz (High-Speed)
Rezistențe pull-up	4.7 kΩ
Pe Arduino Uno	A4 = SDA, A5 = SCL

4.2 Dispozitive pe Magistrala I2C

Dispozitiv	Adresă I2C	Rol	Tensiune
APDS-9930	0x39	Senzor lumină + proximitate	3.3V
LCD 16×2	0x27	Afișaj HMI local	5V

4.3 Structura Mesajului I2C

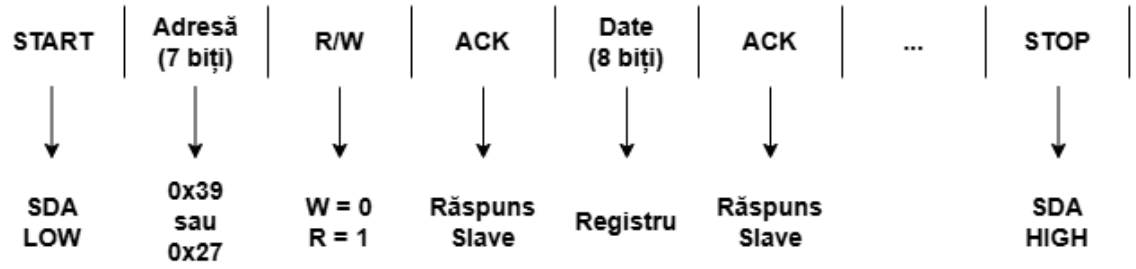


Figura 3: Structura mesajului I2C

5. Arhitectura Algoritmului de Control

5.1. Introducere în Structura de Reglare

Pentru asigurarea și menținerea unui nivel optim de iluminat la valoarea de referință prestabilită, s-a proiectat și implementat un sistem de control în buclă închisă. Având în vedere caracterul neliniar al elementului de execuție (LED-ul alimentat prin intermediul unui potențiometru digital), o structură clasică de reglare liniară ar fi fost ineficientă, conducând fie la oscilații, fie la un răspuns extrem de lent.

Pentru depășirea acestor limitări hardware, arhitectura software dezvoltată este una hibridă și cuprinde două mari module:

1. Regulatorul PID Adaptiv cu Gain Scheduling și Zonă Moartă – responsabil cu calculul comenzii ideale în domeniul liniar.

2. Modulul de Compensare a Neliniarităților – responsabil cu traducerea comenzii ideale în trepte fizice de potențiometru, adaptate la realitatea fizică a circuitului.

5.2. Regulatorul PID Adaptiv

Algoritmul de reglare implementat extinde structura clasică PID prin adăugarea unor strategii moderne de conducere a proceselor. Fluxul de calcul din cadrul funcției de reglare urmează etapele descrise mai jos:

5.2.1. Protecția la Saturație: o problemă critică în sistemele fotometrice o reprezintă orbirea sau saturația senzorului de lumină (de exemplu, sub acțiunea directă a unei surse externe puternice, cum ar fi o lanternă sau razele solare). În această stare, senzorul APDS-9930 raportează o valoare de eroare codificată prin numărul -1. Algoritmul interceptează acest cod și forțează valoarea curentă la o valoare artificială maximă de 1000 lux:

Dacă Lux_curent < 0 => Lux_curent = 1000 lux

5.2.2. Zona Moartă de Eroare (Error Dead-zone): pentru a crește durata de viață a potențiometrului digital X9C103S și a evita fenomenul de jitter, s-a introdus o zonă moartă de 1.5 lux:

Dacă |Eroare| < 1.5 => Eroare = 0.0

Când sistemul se află în toleranța acceptată față de referință, eroarea este anulată software, iar potențiometrul își îngheață poziția actuală.

5.2.3. Strategia de Gain Scheduling Continuu: inovația principală a algoritmului constă în adaptabilitatea coeficienților PID (Kp, Ki, Kd). În loc de o comutare binară a modului de reglare, s-a optat pentru o comutare continuă controlată de un factor adaptiv:

factor_adaptiv = minim(|Eroare| / prag_comutare , 1.0)

Pe baza acestui factor (cuprins între 0 și 1), coeficienții sunt calculați dinamic la fiecare iterație între un mod Conservator (folosit pentru reglaj stabil, fără depășiri) și un mod Agresiv (folosit pentru reducerea rapidă a erorii în cazul perturbațiilor mari):

*Kp = Kp_cons + factor_adaptiv * (Kp_ag - Kp_cons)*

*Ki = Ki_cons + factor_adaptiv * (Ki_ag - Ki_cons)*

*Kd = Kd_cons + factor_adaptiv * (Kd_ag - Kd_cons)*

După determinarea coeficienților optimi pentru starea curentă, se calculează cele trei componente ale legii PID.

Spre deosebire de regulatoarele de poziție standard, acest algoritm folosește o lege de reglare incrementală (de viteză). Suma acțiunilor *(P + I + D)* reprezintă variația necesară, care este adunată la starea anterioară a variabilei de comandă, rezultatul fiind plafonat în intervalul [0,99]:

variabila_comanda(t) = constrain(variabila_comanda(t-1) + (P + I + D) , 0.0 , 99.0)

5.3. Compensarea Neliniarităților

Mărimea de comandă ideală calculată anterior (u) se află într-un domeniu liniar abstract. Din cauza fizicii semiconductoarelor, LED-ul prezintă o zonă moartă electrică la pornire și o curbă de emisie luminoasă neliniară în raport cu tensiunea aplicată. Algoritmul corectează aceste abateri prin trei pași matematici:

5.3.1. Normarea Semnalului: variabila de comandă liniară este normalizată, fiind adusă sub formă de fracție cuprins în intervalul [0, 1]:

u_normat = u / 99.0

5.3.2. Corecția Gamma: senzorii fotometrici sunt mult mai sensibili la variațiile mici de lumină pe întuneric decât la variațiile mari în medii puternic iluminate. Pentru a liniariza răspunsul citit, semnalul normalizat este trecut printr-o funcție de putere cu exponentul gamma = 1.5:

u_nou = (u_normat)^gamma

Această operație introduce o atenuare în prima jumătate a intervalului, obligând potențiometrul digital să execute pași mai lenți în zona de luminozitate scăzută, oferind sistemului o precizie de control superioară acolo unde senzorul este cel mai sensibil.

5.3.3. Compensarea Zonei Moarte Hardware: un LED nu emite lumină până când nu este depășită tensiunea sa de deschidere (forward voltage). Prin definirea parametrului *prag = 5.0* (stabilită empiric), s-a mapat matematic treapta minimă de pornire electrică a componentelor. Valoarea de comandă nouă (aflată între 0 și 1) este re-calibrată prin mapare directă pe intervalul hardware util:

*comanda_noua = prag + u_nou * (99.0 - prag)*

La final, rezultatul obținut este convertit la tipul de date întreg (int), reprezentând numărul fix al treptei fizice la care potențiometrul digital X9C103S va fi mutat prin funcția de execuție nativă *potGoTo*.

6. Arhitectura SCADA

6.1 Nivelurile SCADA ale Sistemului

Sistemul respectă structura pe trei niveluri specifică arhitecturii SCADA (Supervisory Control And Data Acquisition):

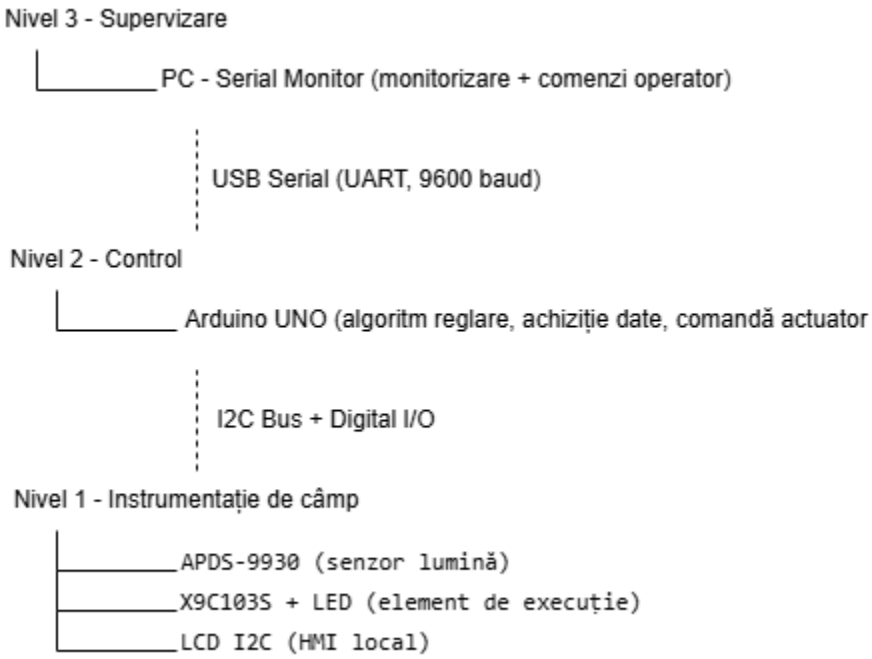


Figura 4: Arhitectura SCADA

6.2 Componente SCADA

Componentă SCADA	**Implementare în Proiect**
Senzor / Traductor	APDS-9930 (lumină → semnal digital I2C)
RTU / PLC	Arduino Uno (achiziție + procesare + comandă)
Element de execuție	X9C103S + LED (comandă digitală → lumină)
HMI local	LCD 16×2 I2C (afișare lux, LED%, proximitate)
HMI remote / SCADA Master	PC prin Serial Monitor (monitorizare + comenzi)
Bus de comunicație	I2C (senzor + LCD), Serial UART (PC)
Protocol	I2C @ 100kHz, UART @ 9600 baud, 8N1

6.3 Funcții SCADA Implementate

Funcție	Detalii
Achiziție date	Citire automată la 0.8 secunde (Ch0, Ch1, proximitate, lux)
Prelucrare	Calcul lux, auto-ranging, legea de reglare
Acționare	Comandă potențiometru digital, ajustare PWM LED
Afișare locală	LCD — lux, nivel LED%, trepte potențiometru
Monitorizare la distanță	Serial Monitor — toate valorile + starea sistemului
Comenzi operator	"l" = citire manuală, "p" = pauză/reluare mod automat
Alarme	Detectare saturare senzor, senzor neconectat

7. Schema Electrică — Tabel de Conexiuni

7.1 Conexiuni Complete

Componentă	Pin Componentă	Pin Arduino	Observații
X9C103S	INC (pin 1)	D3	Increment
X9C103S	U/D (pin 2)	D4	Direcție
X9C103S	VH (pin 3)	5V	Terminal High
X9C103S	VSS (pin 4)	GND	Masă
X9C103S	VW (pin 5)	A0	Wiper → ADC
X9C103S	VL (pin 6)	GND	Terminal Low
X9C103S	CS (pin 7)	D5	Chip Select
X9C103S	VCC (pin 8)	5V	Alimentare
APDS-9930	VCC	3.3V	Alimentare
APDS-9930	GND	GND	Masă
APDS-9930	SDA	A4	I2C Date
APDS-9930	SCL	A5	I2C Ceas
LCD I2C	VCC	5V	Alimentare
LCD I2C	GND	GND	Masă
LCD I2C	SDA	A4	I2C Date (partajat)
LCD I2C	SCL	A5	I2C Ceas (partajat)
LED	Anod (+)	D9 prin 220Ω	PWM
LED	Catod (−)	GND	Masă

7.2 Configurația pinilor și conexiunile hardware

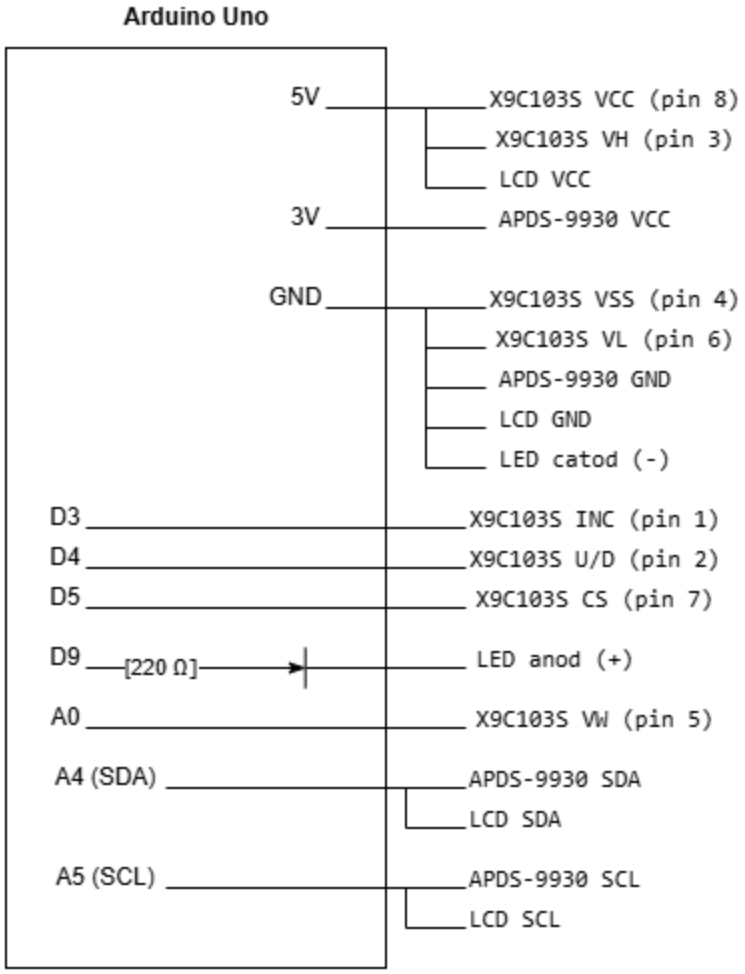


Figura 5: Configurația pinilor și conexiunile hardware

8. Rezultate Testare

Validarea sistemului automat de reglare s-a realizat prin teste directe pe hardware. Deoarece caracteristicile LED-ului și ale potențiometrului digital X9C103S sunt neliniare, acordarea parametrilor PID (pentru modurile conservator și agresiv) a fost efectuată empiric, prin încercări repetate, urmărindu-se un compromis între viteza de răspuns și stabilitate.

În regim tranzitoriu, la modificarea treaptă a referinței, sistemul prezintă un suprareglaj de maximum 5%. În faza finală de convergență, se observă o stabilizare lentă către valoarea dorită. Acest fenomen este cauzat de rezoluția finită a potențiometrului (100 de trepte discrete): la erori mici (2-3 lux), componenta integrativă (Ki) acumulează valori reduse în timp, comutând treptele hardware mai rar.

Robustețea algoritmului adaptiv a fost evaluată prin trei scenarii de testare:

- La 100 lux: sistemul compensează zona moartă electrică a LED-ului și stabilizează intensitatea luminoasă într-un timp scurt.
- La 200 lux (regim nominal): sistemul pornește în modul agresiv și comută treptat pe cel conservator, pe măsură ce eroare scade, atingând referința în banda de ±2% fără oscilații sau instabilitate.
- La 250 lux (saturație hardware): LED-ul aflat în montaj nu posedă puterea fizică necesară pentru a genera un flux atât de puternic pe suprafața senzorului APDS-9930, în condiții fără perturbații externe. Algoritmul PID a incrementat comanda până la pragul maxim (treapta 99), obținându-se un nivel stabil de maximum 245 lux. Această eroare staționară de 5 lux este o limitare strict hardware și nu o eroare de algoritm de control.

9. Concluzii

Proiectul demonstrează implementarea cu succes a unui sistem SCADA simplu, dar pe deplin funcțional, destinat reglării automate și eficiente a iluminării ambientale. Arhitectura propusă integrează toate nivelurile specifice automatizărilor industriale:

- Nivel de câmp: Senzor APDS-9930 (achiziție) + X9C103S/LED (execuție)
- Nivel de control: Arduino Uno cu algoritm de reglare proporțională
- Nivel de supervizare: LCD local + interfață Serial (PC)

Prin corelarea acestor trei niveluri, sistemul a demonstrat o capacitate remarcabilă de adaptare la condițiile variabile de mediu, reușind să mențină referința impusă în limitele de performanță. Integrarea filtrării neliniare a permis liniarizarea comportamentului fizic al LED-ului, transformând sistemul într-o buclă de control robustă. În ciuda limitărilor fizice de putere întâmpinate la praguri ridicate (saturația la 245 lux), algoritmul software a dovedit stabilitatea întregului sistem, împiedicând apariția oscilațiilor dăunătoare elementelor de execuție.

Îmbunătățiri posibile:

- Adăugare modul WiFi (ESP8266/ESP32) pentru monitorizare web reală
- Înregistrare date pe card SD (data logging) pentru analiză ulterioară
- Adăugare mai mulți senzori pe bus-ul I2C pentru monitorizare multi-zonă
- Interfață SCADA industrială (Modbus RTU/TCP) pentru integrare în sisteme existente

10. Bibliografie

[1] Broadcom, „APDS-9930 Digital Proximity and Ambient Light Sensor Datasheet", 2013.

[2] Renesas (Intersil), „X9C102/103/104/503 Digitally Controlled Potentiometer Datasheet", 2005.

[3] Atmel, „ATmega328P Datasheet", 2015.

[4] NXP Semiconductors, „I2C-bus Specification and User Manual", UM10204, Rev. 7.0, 2021.

[5] Hitachi, „HD44780U LCD Controller Datasheet", 1998.

[6] Arduino, „Arduino Uno Rev3 Documentation", Arduino LLC, 2026. https://docs.arduino.cc/hardware/uno-rev3.

[7] Bailey, D., Wright, E., „Practical SCADA for Industry", Elsevier, 2003.

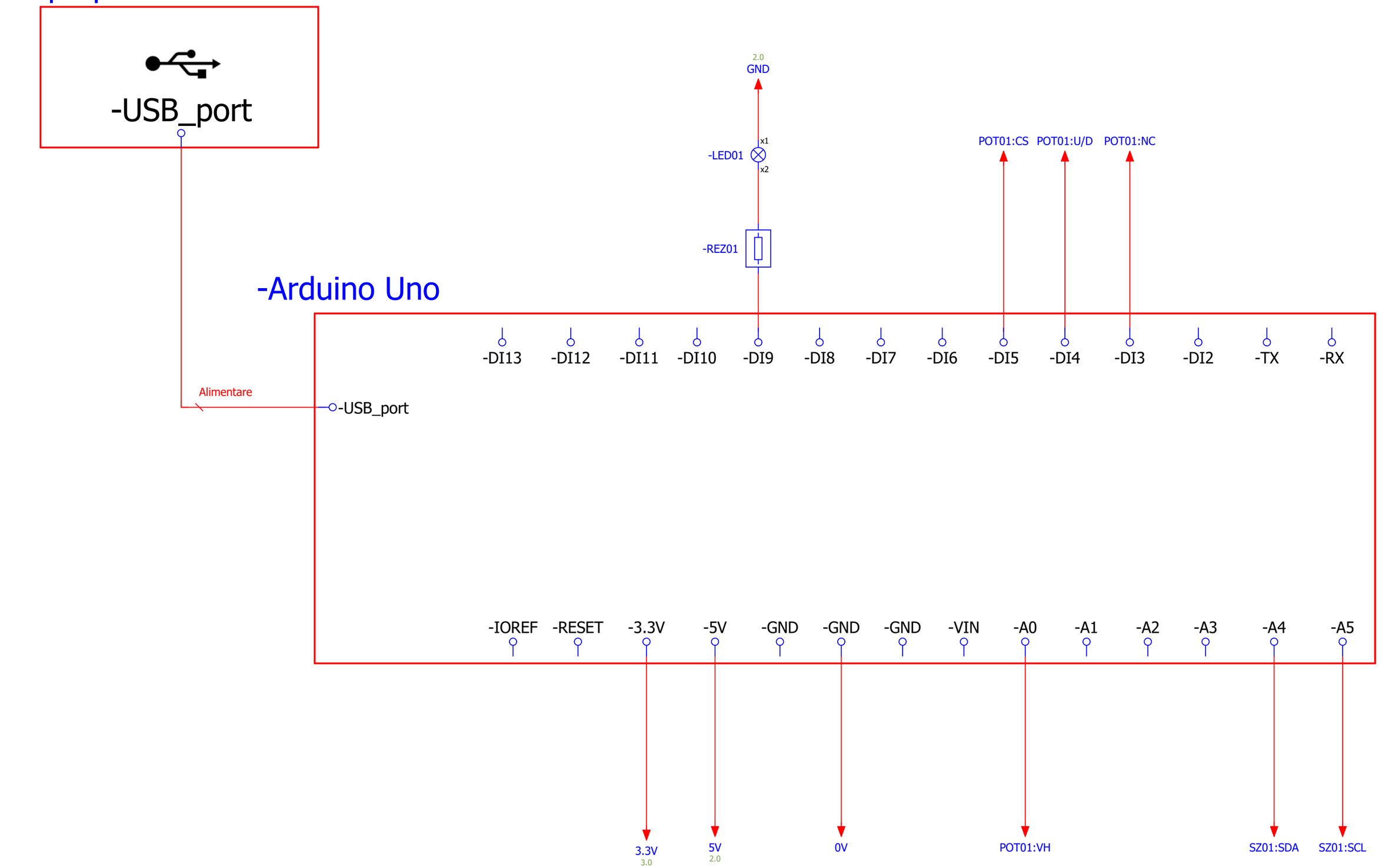
[8] Bolton, W., „Instrumentation and Control Systems", Elsevier, 2015.

[9] K. J. Åström și T. Hägglund, „Advanced PID Control”, Research Triangle Park, NC: ISA - International Society of Automation, 2006.

[10] K. Ogata, „Modern Control Engineering”, 5th ed., Boston, MA: Prentice Hall, 2010.

[11] C. Poynton, „Digital Video and HD: Algorithms and Interfaces”, Waltham, MA: Morgan Kaufmann, 2012.

ANEXA I
Schema electrică



			Data	5/25/2026	v1		Arduino	
			Ed.	.				
			Ver.					Filă 1
Modificare	Data	Nume	Orig.		Înlocuit cu	Înlocuit de		Pagină 1 / 3

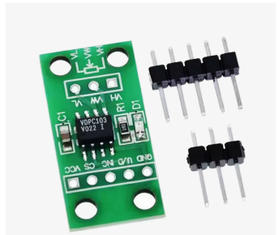
1.4 / 5V

1.5 / GND

5V /

GND / 3.0

-POT01
X9C103S



A0

-POT01:VCC -POT01:VL -POT01:VH

-POT01:VCC -POT01:CS -POT01:NC -POT01:U/D -POT01:GND

DI5 DI4 DI3

1

3

			Data	5/25/2026	v1		Potentiometru	
			Ed.					
			Ver.					Filă 2
Modificare	Data	Nume	Orig.		Înlocuit cu	Înlocuit de		Pagină 2 / 3

1.4 / 3.3V

2.9 / GND

3.3V /

GND /

-SZ01
APDS-9930



A5 A4

-REZ02 -REZ03

-SZ01:VCC -SZ01:GND -SZ01:SCL -SZ01:SDA

2

			Data	5/25/2026	v1		SZ01		
			Ed.						
			Ver.						Filă 3
Modificare	Data	Nume	Orig.		Înlocuit cu	Înlocuit de			Pagină 3 / 3

ANEXA II
Codul sursă

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>

// ---- LCD I2C ----
LiquidCrystal_I2C lcd(0x27, 16, 2);

// ---- APDS-9930 ----
#define APDS_ADDR     0x39 // adresa senzorului pe magistrala
#define APDS_ENABLE   0x80
#define APDS_ATIME    0x81
#define APDS_PTIME    0x82
#define APDS_WTIME    0x83
#define APDS_PPULSE   0x8E
#define APDS_CONTROL  0x8F
#define APDS_ID       0x92
#define APDS_CH0DATAL 0x94
#define APDS_CH1DATAL 0x96
#define APDS_PDATAL   0x9C

uint8_t currentATIME = 0xED; // 51 ms de captura a luminii
uint8_t currentAGAIN = 0; // treapta de sensibilitate (din datasheet)
const float againValues[] = {1.0, 8.0, 16.0, 120.0}; // valorile in functie de treapta

// ---- X9C103S ----
const int CS_PIN  = 5; // pin D5
const int UD_PIN  = 4; // pin D4
const int INC_PIN = 3; // pin D3

// ---- LED & Wiper ----
const int LED_PIN   = 9; // pin D9 PWM
const int WIPER_PIN = A0; // citirea tensiunii de la potentiometru

// ---- Timer ----
unsigned long lastReadTime = 0;
const unsigned long READ_INTERVAL = 800; // ms perioada de citire si afisare date
int currentPosition = 0; // treapta initiala a potentiometrului
bool autoPaused = false;

// ========================================
```

```
// ====== REGULATOR PID ADAPTIV =============
// ==========================================
const float referinta_lux = 200.0;

float Kp, Ki, Kd;

// Mod CONSERVATOR
const float Kp_cons = 0.045;
const float Ki_cons = 0.014;
const float Kd_cons = 0.690;

// Mod AGRESIV
const float Kp_ag = 0.095;
const float Ki_ag = 0.004;
const float Kd_ag = 0.410;

const float prag_comutare = 40.0;

// Variabile PID
float suma_eroare = 0;
float eroare_trecuta = 0;
const float limita_integrare = 20.0;

const int MAX_STEP = 5; // variatia maxima a treptelor potentiometrului la o iteratie
bool mod = false; // pentru afisare

// variabila_comanda reprezinta cererea ideala a PID-ului, nu pozitia hardware
float variabila_comanda = 50.0;
float lux_filtrat = 0;
const float alpha = 0.5; // coeficient de filtrare pe iesire

// ==========================================
byte barChar[8] = {
  0b11111, 0b11111, 0b11111, 0b11111,
  0b11111, 0b11111, 0b11111, 0b11111 };

// ==========================================
// --- APDS-9930 ---
// ==========================================
```

```
void apdsWrite(uint8_t reg, uint8_t val) {
  Wire.beginTransmission(APDS_ADDR);
  Wire.write(reg); Wire.write(val);
  Wire.endTransmission(); }

uint8_t apdsRead(uint8_t reg) {
  Wire.beginTransmission(APDS_ADDR);
  Wire.write(reg); Wire.endTransmission();
  Wire.requestFrom((uint8_t)APDS_ADDR, (uint8_t)1);
  return Wire.read(); }

uint16_t apdsRead16(uint8_t reg) {
  uint8_t lo = apdsRead(reg);
  uint8_t hi = apdsRead(reg + 1);
  return (hi << 8) | lo; }

float getALSIT() { return 2.73f * (256 - currentATIME); }

void setAGAIN(uint8_t gain) {
  currentAGAIN = constrain(gain, 0, 3);
  uint8_t ctrl = apdsRead(APDS_CONTROL);
  ctrl = (ctrl & 0xFC) | currentAGAIN;
  apdsWrite(APDS_CONTROL, ctrl); }

void setATIME(uint8_t atime) {
  currentATIME = atime;
  apdsWrite(APDS_ATIME, atime); }

void autoRange(uint16_t ch0) {
  if (ch0 >= 65000) {
    if (currentAGAIN > 0) setAGAIN(currentAGAIN - 1);
    else if (currentATIME < 0xFE) setATIME(currentATIME + 10);
  } else if (ch0 < 100 && ch0 > 0) {
    if (currentAGAIN < 3) setAGAIN(currentAGAIN + 1);
    else if (currentATIME > 0xC0) setATIME(currentATIME - 10);
  }
}
```

```
bool apdsInit() {
  uint8_t id = apdsRead(APDS_ID);
  if (id != 0x39 && id != 0x12 && id != 0x30) return false;

  apdsWrite(APDS_ENABLE, 0x00); delay(10);
  apdsWrite(APDS_ATIME, currentATIME);
  apdsWrite(APDS_PTIME, 0xFF);
  apdsWrite(APDS_WTIME, 0xFF);
  apdsWrite(APDS_PPULSE, 0x08);
  apdsWrite(APDS_CONTROL, 0x20 | currentAGAIN);
  apdsWrite(APDS_ENABLE, 0x0F); delay(12);
  return true; }

uint16_t getAmbientLight() { return apdsRead16(APDS_CH0DATAL); }
uint16_t getIRLight()      { return apdsRead16(APDS_CH1DATAL); }
uint16_t getProximity()    { return apdsRead16(APDS_PDATAL); }

float calculateLux(uint16_t ch0, uint16_t ch1) {
  if (ch0 >= 65000) return -1;
  float iac1 = (float)ch0 - 1.862f * (float)ch1;
  float iac2 = 0.746f * (float)ch0 - 1.291f * (float)ch1;
  float iac  = max(max(iac1, iac2), 0.0f);
  float lpc  = (0.49f * 52.0f) / (getALSIT() * againValues[currentAGAIN]);
  return iac * lpc; }

// ========================================
// --- X9C103S ---
// ========================================

void potStep(bool up) {
  digitalWrite(CS_PIN, LOW);
  digitalWrite(UD_PIN, up ? HIGH : LOW); delayMicroseconds(1);
  digitalWrite(INC_PIN, LOW); delayMicroseconds(1);
  digitalWrite(INC_PIN, HIGH); delayMicroseconds(1);
  digitalWrite(CS_PIN, HIGH); delay(1); }

void potMove(bool up, int steps) {
  for (int i = 0; i < steps; i++) potStep(up); }
```

```
void potReset() {
  potMove(false, 100);
  currentPosition = 0; }

void potGoTo(int targetPos) {
  targetPos = constrain(targetPos, 0, 99);
  int dif = targetPos - currentPosition;
  if (dif > 0) potMove(true, dif);
  else if (dif < 0) potMove(false, -dif);
  currentPosition = targetPos; }

// ========================================

// ========================================
// --- Algoritm PID Adaptiv  ---
// ========================================

float legeReglare(float lux_curent) {
  if (lux_curent < 0) lux_curent = 1000; // in caz de saturatie a senzorului

  float eroare = referinta_lux - lux_curent;
  float abs_eroare = abs(eroare);

  // Zona moarta pentru eroare
  // Previne actiunea potentiometrului digital cand eroarea este mai mica decat 1.5 lux
  if (abs_eroare < 1.5) {
    eroare = 0.0; }

  // Gain scheduling
  // Calculam un factor de tranzitie intre 0 (conservator) si 1 (agresiv)
  float factor_adaptiv = abs_eroare / prag_comutare;
  if (factor_adaptiv > 1.0) factor_adaptiv = 1.0;

  // Combinam ambele moduri de reglare in functie de marimea erorii
  Kp = Kp_cons + factor_adaptiv * (Kp_ag - Kp_cons);
  Ki = Ki_cons + factor_adaptiv * (Ki_ag - Ki_cons);
  Kd = Kd_cons + factor_adaptiv * (Kd_ag - Kd_cons);

  mod = (abs_eroare > prag_comutare);
```

```
  // Proportional
  float P = Kp * eroare;

  // Integrativ
  suma_eroare += eroare;
  suma_eroare = constrain(suma_eroare, -limita_integrare, limita_integrare); // anti-windup simplu
  float I = Ki * suma_eroare;

  // Derivativ
  float derivata_eroare = eroare - eroare_trecuta;
  float D = Kd * derivata_eroare;
  eroare_trecuta = eroare;

  // -Actualizare comanda
  variabila_comanda += (P + I + D);
  variabila_comanda = constrain(variabila_comanda, 0.0, 99.0);

  return variabila_comanda; // variabila de comanda (0-99 trepte) }


// ========================================
// --- COMPENSARE NELINIARITATI ---
// ========================================

// Aceasta sectiune de cod face legatura intre domeniul matematic liniar al PID-ului
// si domeniul fizic neliniar al LED-ului comandat de potentiometrul digital
// Transformarea prin exponentiala compenseaza faptul ca LED-ul tinde sa fie mult prea luminos la inceputul cursei (x << 1)

int compensareNeliniaritati(float u) {
  if (u < 0.5) return 0; // daca comanda este apropiata de 0, sarim peste compensare si dam comanda nula

  const float prag = 5.0; // treapta hardware minima de la care LED-ul incepe sa lumineze (aleasa empiric)

  const float gamma = 1.5; // coeficient de curbare

  float u_normat = u / 99.0; // normarea variabilei de comanda
  float u_nou = pow(u_normat, gamma); // aplicam curba y = x^gamma
```

```
  float comanda_noua = prag + (u_nou * (99.0 - prag)); // mapare inapoi pe treptele potentiometrului

  return constrain((int)comanda_noua, 0, 99); }
// ========================================

// ========================================
// --- Afisare LCD ---
// ========================================

void updateLCD(float lux, int intensitate_LED) {
  lcd.setCursor(0, 0);
  if (lux < 0) {
    lcd.print("Lux: SATURAT    ");
  } else {
    lcd.print("Lux: "); lcd.print((int)lux); lcd.print(" lx     "); }

  lcd.setCursor(0, 1);
  lcd.print("LED:"); lcd.print(map(intensitate_LED, 0, 255, 0, 100)); lcd.print("%  ");

  lcd.setCursor(8, 1);
  lcd.print("Pot:"); lcd.print(map(currentPosition, 0, 99, 0, 100)); lcd.print("%  "); }

// ========================================
// --- Citire + reglare ---
// ========================================

void doAutoRead() {
  uint16_t ch0 = getAmbientLight(), ch1 = getIRLight();
  float lux = calculateLux(ch0, ch1);

  lux_filtrat = alpha * lux + (1 - alpha) * lux_filtrat;

  // PID-ul calculeaza variabila de comanda (liniara)
  float u_pid = legeReglare(lux_filtrat);

  // Trecem variabila prin filtrul neliniar pentru a afla comanda hardware
  int targetPos = compensareNeliniaritati(u_pid);

  int dif = targetPos - currentPosition;
```

```
  if (dif > MAX_STEP) dif = MAX_STEP;
  if (dif < -MAX_STEP) dif = -MAX_STEP;

  potGoTo(currentPosition + dif);

  int intensitate_LED = map(currentPosition, 0, 99, 0, 255); // calculeaza lumina emisa de LED in
functie de treapta potentiometrului (% din maxim)

  updateLCD(lux, intensitate_LED);

  Serial.println("====== PID ADAPTIV CU COMPENSARE NELINIARITATI ======");
  Serial.print("  Lux (Filtrat): ");
  if (lux < 0) Serial.println("ERR"); else Serial.println(lux_filtrat, 1);
  Serial.print("  Mod adaptiv: "); Serial.println(mod ? "AGRESIV" : "CONSERVATOR");
  Serial.print("  LED: "); Serial.print(map(intensitate_LED, 0, 255, 0, 100)); Serial.println("%");
  Serial.print("  Potentiometru: "); Serial.print(currentPosition); Serial.println("/99");
  Serial.println("=====================================");

  autoRange(ch0);}

// ========================================
// --- Setup + Loop ---
// ========================================
void setup() {
  pinMode(CS_PIN, OUTPUT); pinMode(UD_PIN, OUTPUT);
  pinMode(INC_PIN, OUTPUT); pinMode(LED_PIN, OUTPUT);

  analogWrite(LED_PIN, 0);

  digitalWrite(CS_PIN, HIGH); digitalWrite(INC_PIN, HIGH);

  Serial.begin(9600); Wire.begin();
  lcd.init(); lcd.backlight(); lcd.createChar(0, barChar);
  lcd.setCursor(0, 0); lcd.print("Initializare...");

  if (!apdsInit()) {
    lcd.clear(); lcd.print("EROARE APDS!");
    Serial.println("EROARE: APDS-9930!");  }
```

```
  potReset();

  // Variabile initiale
  uint16_t ch0 = getAmbientLight(), ch1 = getIRLight();
  lux_filtrat = calculateLux(ch0, ch1);
  eroare_trecuta = referinta_lux - lux_filtrat;

  lcd.clear(); lcd.setCursor(0, 0); lcd.print("Gata! Auto");

  lastReadTime = millis();
  delay(2000);}

void loop() {
  // Calculam luminozitatea din pozitia curenta a potentiometrului, eliminand zgomotul senzorului
  int intensitate_LED = map(currentPosition, 0, 99, 0, 255);
  analogWrite(LED_PIN, intensitate_LED);

  // Apelare afisare date
  if (!autoPaused && (millis() - lastReadTime >= READ_INTERVAL)) {
    lastReadTime = millis();
    doAutoRead(); }

  // Definim doua comenzi input pe serial: fortare citire si pauza afisare
  if (Serial.available()) {
    char c = Serial.read();
    if (c == 'l') {
      doAutoRead(); lastReadTime = millis();
    } else if (c == 'p') {
      autoPaused = !autoPaused;
      lcd.clear(); lcd.setCursor(0, 0);
      lcd.print(autoPaused ? "AUTO: PAUZA" : "AUTO: ACTIV"); } }
  delay(50);}
```